Manual de uso - Llama Desktop Launcher

Versão 0.9.0 · Português (Brasil)

Este é o primeiro de três guias. O **Guia de integração** (INTEGRATION-pt-BR) explica como conectar OpenCode, outros clientes e seus próprios aplicativos à API. O **Guia de diagnóstico** (DIAGNOSTICS-pt-BR) reúne problemas comuns, mensagens do aplicativo e o relatório para suporte. Os três estão no rodapé do aplicativo e na pasta docs/ do repositório.

1. O que o aplicativo faz

O Llama Desktop Launcher prepara e inicia um servidor de modelos de linguagem no seu computador e o expõe como uma API compatível com OpenAI. Ele trabalha com três motores:

- **[llama.cpp](https://github.com/ggml-org/llama.cpp)** (versão testada b12211) para arquivos **GGUF**. É o motor principal: tem conversa integrada, CLI no terminal, sugestão de ajustes e benchmark.
- **[vLLM](https://github.com/vllm-project/vllm)** 0.32.0 para **pastas Hugging Face** (config.json e pesos .safetensors, incluindo AWQ e GPTQ). Roda no Linux por um ambiente Python do launcher e no Windows pelo Docker Desktop.
- **[Strata](https://github.com/Niko1221/Strata) e [PolyStrata](https://github.com/simplexai-labs/PolyStrata)** para os modelos que esses projetos instalam. Você instala com o instalador oficial deles, e o launcher só inicia o servidor.

Você escolhe o acesso: somente este computador, rede interna (LAN) ou Tailscale. Fora do próprio computador, a API exige uma chave, e você pode compartilhar o modelo com links e comandos que podem ser revogados.

O instalador do aplicativo **não inclui modelos nem motores**. O llama.cpp é baixado pelo próprio aplicativo; o vLLM e o Strata têm preparação própria (seções 6 e 7).

A interface tem três telas, acessíveis pela navegação no topo:

- **Llama:** escolher o modelo, iniciar o servidor, conversar e copiar as configurações dos clientes.
- **Modelos:** recomendações pelo seu hardware e links para encontrar modelos de cada motor.
- **Configurações:** motores, pastas de modelos, perfis, importação, atualizações, diagnóstico e, no Windows, as regras de firewall criadas pelo launcher.

A versão do launcher aparece no cabeçalho, ao lado de “llama.cpp no seu computador”. A versão de cada motor é independente.

2. Requisitos

Sistema	Arquitetura	llama.cpp	vLLM	Strata / PolyStrata
Windows 10/11	x64	Vulkan, NVIDIA CUDA, AMD ROCm, Intel SYCL ou CPU	Docker Desktop com WSL2 e GPU NVIDIA	Sim, pelo instalador oficial deles
Linux (DEB, RPM, AppImage)	x64	Vulkan, CUDA, ROCm, SYCL ou CPU	Python 3.10-3.13 e GPU NVIDIA (CUDA)	Sim, pelo instalador oficial deles
Linux (DEB, RPM, AppImage)	ARM64	Vulkan ou CPU	Não testado	Conforme o projeto
macOS (Apple Silicon e Intel)	ARM64 / x64	Metal	Não oferecido	Não oferecido

O espaço necessário depende do modelo. Reserve espaço para os pesos, para o cache e para o motor (o pacote CUDA do llama.cpp e a imagem Docker do vLLM têm vários GB). Modelos grandes podem precisar de muito mais memória do que o tamanho em disco. Sem GPU compatível, o llama.cpp cai para CPU e fica lento; o vLLM e o Strata exigem GPU. No Linux, o pacote oficial do llama.cpp é compilado no Ubuntu; outras distribuições podem precisar de bibliotecas compatíveis.

3. Instalar o aplicativo

Os instaladores e o arquivo de verificação SHA256SUMS.txt estão na página da versão mais recente. O código-fonte está no repositório do projeto.

Sistema	Arquivo na release (X.Y.Z = versão)
Windows x64	Llama.Desktop.Launcher.Setup.X.Y.Z.exe
Debian, Ubuntu, Mint, Pop!_OS	llama-desktop-launcher_X.Y.Z_amd64.deb ou _arm64.deb
Fedora, RHEL, openSUSE	llama-desktop-launcher-X.Y.Z.x86_64.rpm ou .aarch64.rpm
Qualquer distribuição Linux (portátil)	Llama.Desktop.Launcher-X.Y.Z.AppImage ou -arm64.AppImage
macOS Apple Silicon	Llama.Desktop.Launcher-X.Y.Z-arm64.dmg
macOS Intel	Llama.Desktop.Launcher-X.Y.Z.dmg

Windows

1. Abra o instalador .exe.
2. Escolha a pasta de instalação. O instalador é por usuário e não precisa de Program Files.
3. Marque ou desmarque o atalho na área de trabalho. Atualizações e a instalação silenciosa (/S) mantêm a escolha anterior.
4. Abra o aplicativo pelo Menu Iniciar ou pelo atalho.

Linux

- **DEB / RPM:** instale pelo gerenciador de pacotes da distribuição (sudo apt install ./arquivo.deb ou sudo dnf install ./arquivo.rpm).
- **AppImage:** torne o arquivo executável (chmod +x) e abra-o. Ele fica na pasta que você escolher.

macOS

1. Abra o DMG e arraste o aplicativo para **Aplicativos** (ou outra pasta).
2. Abra pelo Finder. Como os pacotes ainda não são assinados nem notarizados, o macOS pode pedir autorização em **Ajustes do Sistema > Privacidade e Segurança**. Autorize só este aplicativo; não desative as proteções do sistema.

4. Primeiro uso

Na primeira abertura, o aplicativo vai para **Configurações** enquanto faltar um motor pronto, um modelo nas pastas ou um modelo selecionado. Quando tudo estiver pronto, as próximas aberturas vão direto para **Llama**. Você pode sair a qualquer momento com **Concluir e ir para Llama** e voltar depois.

Roteiro sugerido:

1. **Prepare um motor.** Para começar, use o llama.cpp (seção 5). vLLM e Strata são opcionais.
2. **Escolha as pastas de modelos** em **02 · Pastas** (seção 8). A primeira começa em ~/Models/GGUF; a segunda é opcional.
3. **Obtenha um modelo.** A tela **Modelos** mostra o que cabe no seu hardware e abre a busca no Hugging Face. Baixe pelo navegador para uma das pastas e clique em **Atualizar lista**.
4. **Selecione o modelo** na tela **Llama** e clique em **Conversar aqui** (GGUF) ou **Iniciar servidor**.

5. Motor llama.cpp

Em **Configurações > 01 · Motor**, escolha uma das opções:

1. **Baixar motor oficial:** escolha o **Backend do pacote oficial** e clique no botão. O aplicativo baixa a release b12211 do ggml-org/llama.cpp no GitHub para a sua plataforma e confere o SHA-256 quando a release informa o resumo. O motor fica em engines/ na pasta de dados do aplicativo, fora da instalação.
2. **Escolher pasta existente:** use uma pasta que já contenha llama-server (ou llama-server.exe), por exemplo a pasta b11193-vulkan do launcher Windows anterior.

Escolha o backend pela GPU, não pela marca do processador:

- **Vulkan:** funciona com GPUs AMD, Intel e NVIDIA com driver atualizado. É a opção mais universal.
- **NVIDIA CUDA:** melhor desempenho em placas NVIDIA. O download inclui as bibliotecas CUDA; basta um driver compatível. Para uma RTX 4060 no Windows, comece por **NVIDIA CUDA 12.4**.
- **AMD ROCm / Intel SYCL:** só em x64, e podem exigir runtimes adicionais.
- **CPU:** sem aceleração. **Metal:** macOS.

O download é extraído em uma pasta temporária ao lado do destino final; se falhar, a pasta é removida e o motor anterior continua configurado. Para trocar de motor, baixe outro backend ou escolha outra pasta; pare o servidor e a conversa antes.

Importar perfis do launcher anterior

Se você usava o launcher WinForms, configure primeiro a pasta dos seus GGUF. Depois, em **Configurações > 04 · Importação**, clique em **Importar perfis TSV** e escolha launcher-profiles.tsv. Os perfis são associados pelo caminho completo do modelo; perfis já salvos não são sobrescritos. O resultado mostra quantos foram importados, não localizados, já existentes ou inválidos.

6. Motor vLLM

O vLLM serve modelos em formato Hugging Face: uma pasta com config.json e pesos .safetensors (inclusive AWQ e GPTQ). Coloque essas pastas dentro das pastas de modelos; elas aparecem na lista com o sufixo · vLLM. Pastas do cache do Hugging Face (models--org--nome/snapshots/<hash>) também são reconhecidas.

Linux

1. Instale python3 e python3-venv (Python 3.10 a 3.13) e o driver NVIDIA com CUDA.
2. Em **Configurações > 01B · Outros motores**, clique em **Instalar vLLM**. O launcher cria um ambiente Python na pasta de dados e roda pip install vllm==0.32.0 (vários GB). O andamento aparece no **Registro do motor**, na tela **Llama**.
3. Se você já tem um ambiente com o vLLM, use **Usar ambiente existente** e escolha a pasta que contém bin/vllm.

O pacote do pip só traz suporte a NVIDIA CUDA. Para AMD (ROCm), instale o vLLM pela documentação oficial e aponte o ambiente pronto.

Windows

1. Instale o Docker Desktop com o backend WSL2 e suporte a GPU NVIDIA, e deixe-o aberto.
2. Em **01B · Outros motores**, clique em **Baixar imagem do vLLM**. O launcher baixa vllm/vllm-openai:v0.32.0 (cerca de 10 GB). Use **Verificar de novo** depois de abrir o Docker Desktop.
3. Ao iniciar o servidor, o modelo é montado no contêiner **só para leitura** e sem acesso à internet (HF_HUB_OFFLINE=1): a pasta precisa estar completa. O contêiner é parado junto com o servidor.

O que vale no vLLM

- O perfil do vLLM tem campos próprios: uso de memória da GPU, tipo do cache KV, paralelismo de tensor, decodificação especulativa e outros (seção 10). As **Sessões simultâneas** viram --max-num-seqs.
- A primeira carga pode levar vários minutos (compilação de grafos). O launcher espera até 30 minutos.
- **Abrir no navegador** abre a documentação da API (/docs), porque o vLLM não tem interface de conversa.
- No macOS o vLLM não é oferecido.

7. Strata e PolyStrata

Strata e PolyStrata servem famílias de modelos específicas, com quantização e uso de GPU próprios. Funcionam no Windows e no Linux.

1. Instale pelo instalador oficial do projeto (Strata ou PolyStrata), com START-HERE.bat (Windows) ou setup.sh (Linux), e configure ao menos um modelo. O instalador deles baixa os pesos e cria a pasta .venv.
2. Em **01B · Outros motores**, clique em **Escolher pasta instalada** e selecione a pasta que contém serve/server.py e .venv.
3. Cada arquivo strata-<modelo>.json dessa pasta vira um item da lista, com o sufixo · Strata ou · PolyStrata.

O launcher não baixa nem executa o instalador desses projetos. Contexto, quantização e GPU vêm da configuração strata-*.json; o launcher define o endereço, a porta e a chave de API. Opções extras do servidor (GPU, fila, descarregar após ociosidade, VRAM mínima) aparecem no perfil quando a versão instalada as aceita (seção 10). As **Sessões simultâneas** do launcher valem só para o Strata; o PolyStrata usa o valor da própria configuração.

Para descobrir modelos, use o cartão **Outros motores** na tela **Modelos**. Ele tem links para as listas do Strata, do PolyStrata e do vLLM e diz se a GPU detectada atende aos requisitos de cada um.

8. Pastas e modelos

O aplicativo aceita **duas pastas de modelos** em **Configurações > 02 · Pastas**. Use **Alterar** para escolher outro disco, **Abrir** para ver a pasta e **Remover** para tirar a segunda. A busca entra nas subpastas.

Motor	O que é reconhecido como modelo	Como aparece na lista
llama.cpp	Arquivos .gguf; em modelos divididos, só o primeiro shard	Nome do arquivo
vLLM	Pastas com config.json e .safetensors	Nome da pasta · vLLM
Strata / PolyStrata	strata-*.json na pasta instalada do Strata	model_name · Strata ou · PolyStrata

Para GGUF:

- Na tela **Modelos**, use **Explorar todos os GGUF**, **Buscar quantizados**, **Buscar MoE** ou **Buscar densos F16** para abrir o Hugging Face no navegador. Q4_K_M é um bom ponto de partida pelo equilíbrio entre memória e qualidade.
- Confira a licença, os requisitos de memória e as instruções do autor antes de baixar.
- Se o modelo usa projetor multimodal, deixe um único mmproj*.gguf na mesma pasta; ele é associado automaticamente.
- Mantenha juntos todos os shards de um modelo dividido.

GGUF descreve o formato, mas não garante compatibilidade com b12211; alguns modelos exigem uma versão mais nova do motor. O aplicativo não aceita pastas de modelos dentro da instalação nem dentro da pasta de dados interna.

As estimativas da tela **Modelos** usam RAM, CPU, GPU/VRAM detectável e espaço livre. São pontos de partida conservadores, não garantia. Para modelos MoE, considere o tamanho total dos pesos, não só os parâmetros ativos.

9. Tela Llama: conversar e iniciar o servidor

1. Escolha o modelo na lista **Modelo**. A linha abaixo mostra o tamanho, o caminho e o motor.
2. Escolha o **Endereço de acesso** e as **Sessões simultâneas** (1 a 8) em **Acesso e sessões**.
3. Use um dos botões:

- **Conversar aqui** (só GGUF): abre o llama-cli dentro da janela. Enter envia; Shift+Enter quebra a linha. O histórico não é salvo.
- **Abrir CLI no terminal** (só GGUF): abre o llama-cli em um terminal separado, com o modelo e os ajustes atuais. No Linux, é preciso ter um terminal gráfico instalado.
- **Iniciar servidor:** carrega o modelo e mostra o endpoint da API, sem abrir o navegador.
- **Abrir no navegador:** inicia, se preciso, e abre a interface do motor. Com o servidor ativo, só abre a página.
- **Parar servidor:** encerra o servidor, o compartilhamento e, no vLLM do Windows, o contêiner.

Só uma execução por vez: pare o servidor, a conversa ou o teste de desempenho antes de iniciar outro. Fechar a aba do navegador não para o servidor; fechar o launcher para.

O **Registro do motor**, abaixo dos botões, mostra a saída do motor e as mensagens do launcher. Ele é a primeira fonte para entender uma falha (veja o Guia de diagnóstico).

O que cada motor oferece

Recurso	llama.cpp	vLLM	Strata / PolyStrata
Conversa integrada e CLI	Sim	Não	Não
Sugestão inicial	Sim	Sim	Não
Autotuning	llama-bench e llama-server curto	Opcional; recarrega o modelo a cada tentativa	Não
Abrir no navegador	Interface web do llama-server	Documentação da API (/docs)	Página do servidor do Strata
Ajustes do perfil	Os do llama.cpp	Os do vLLM	Sessões (só Strata) e opções que o server.py instalado aceitar
Espera máxima pela carga	5 minutos	30 minutos	30 minutos
ID do modelo na API	modelo-local	modelo-local	model_name da configuração

10. Perfis por modelo

Cada modelo tem um perfil próprio, salvo pelo caminho do arquivo ou da pasta. O perfil é salvo quando você clica em **Salvar ajustes deste modelo** e também quando inicia um servidor ou uma conversa com aquele modelo.

Na tela **Llama**, os ajustes ficam no acordeão **Ajustes avançados deste modelo**, fechado no início. Em **Configurações > 03 · Perfil do modelo**, os mesmos campos aparecem abertos, com um seletor de modelo.

Como o painel funciona

- **Só aparecem os parâmetros do motor do modelo escolhido.** Um GGUF mostra os do llama.cpp; uma pasta Hugging Face, os do vLLM; uma configuração strata-*.json, os do Strata.
- **O que o modelo ou o hardware não suporta fica recolhido** em **Não se aplicam a este modelo (N)**, cada um com o motivo. Exemplos: "Modelo denso: não tem especialistas MoE", "Só com 2 ou mais GPUs", "O modelo não usa atenção com janela deslizante (SWA)".
- **Cada campo tem a caixa Auto.** Marcada, o launcher não passa a opção e o motor decide. Abaixo do campo aparece o valor automático quando dá para saber, com a origem: Automático: 25 · ajuste automático (fit).
- As etiquetas no topo resumem o que o launcher leu do modelo: arquitetura, camadas, contexto de treino e tipo (denso, MoE, SWA, híbrido/recorrente, MTP).

O launcher descobre o que o modelo suporta sem carregar os pesos: lê o cabeçalho do GGUF (especialistas, camadas MTP/NextN, janela deslizante, camadas recorrentes), o config.json da pasta Hugging Face (especialistas, MTP, quantização, contexto máximo, cabeças de atenção) e as GPUs (llama-server --list-devices no llama.cpp, nvidia-smi no vLLM e no Strata).

llama.cpp (GGUF)

Grupo	Campo	Argumento	Aparece quando
GPU e memória	Camadas na GPU	-ngl	Há GPU. Valor: número, ou all.
	Contexto	-c	Sempre. Em Auto, o -fit escolhe o maior que cabe.
	Contexto mínimo do ajuste automático	-fitc	Contexto em Auto.
	Margem livre por GPU (MiB)	-fitt	Há GPU.
	Dispositivo	-dev	Há GPU. Nomes do --list-devices (ex.: CUDA0, Vulkan1); vários separados por vírgula; none = só CPU.
	Divisão entre GPUs / Proporção por GPU	-sm / -ts	2 ou mais GPUs.
	Cache KV fora da GPU	-nkvo	Há GPU.
	Modo de carga	-lm	Sempre: auto, none, mmap, mlock, mmap+mlock, dio.
Especialistas MoE	Camadas de especialistas na CPU	-ncmoe	Modelo MoE.
	Cache de especialistas na GPU (MiB)	--moe-cache-mib	Modelo MoE com GPU.
	Fração das faltas do cache feita na CPU	--moe-cache-cpu-share	Cache de especialistas maior que 0.
Modelo denso	Camadas com FFN na CPU	-ncffn	Modelo sem especialistas, com GPU.
CPU e lotes	Threads de geração / de prompt	-t / -tb	Sempre.
	Batch / Microbatch	-b / -ub	Sempre. O microbatch não pode passar do batch.
Cache KV e atenção	Cache K / V	-ctk / -ctv	Sempre. Cache V quantizado exige Flash Attention.
	Flash Attention	-fa	Sempre.
	Cache SWA completo	--swa-full	Modelo com janela deslizante.
	Checkpoints de contexto por sessão	-ctxcp	Modelo SWA, híbrido ou recorrente.
	Reuso de cache por deslocamento	--cache-reuse	Só no servidor.
	Cache KV unificado entre sessões	-kvu	Servidor com 2 ou mais sessões simultâneas.

Grupo	Campo	Argumento	Aparece quando
Decodificação especulativa	Tipo	--spec-type	Sempre: none, ngram-mod, ngram-simple, ngram-map-k; draft-mtp só se o GGUF tiver camadas MTP; draft-simple/draft-eagle3 com um modelo rascunho.
	Modelo rascunho	-md	Tipos draft-simple e draft-eagle3. Lista os GGUF menores das pastas.
	Tokens por rascunho	--spec-draft-n-max	Tipos draft-*.

O launcher sempre passa -fit on: com campos em Auto, o llama.cpp ajusta camadas, contexto e especialistas na CPU à memória livre. **Sessões simultâneas** (--parallel) dividem o contexto: com contexto 8192 e 2 sessões, cada cliente tem 4096 tokens.

vLLM (pasta Hugging Face)

Campo	Argumento	Aparece quando
Contexto	--max-model-len	Sempre. Em Auto, vale o max_position_embeddings do modelo.
Uso de memória da GPU	--gpu-memory-utilization	Sempre (0,05 a 0,99; padrão 0,9).
Tipo do cache KV	--kv-cache-dtype	Sempre: auto, float16, bfloat16, fp8, fp8_e4m3, fp8_e5m2. fp8 depende da GPU.
Pesos na RAM / Cache KV na RAM (GiB)	--cpu-offload-gb / --kv-offloading-size	Sempre.
Tokens por lote	--max-num-batched-tokens	Sempre.
Cache de prefixo	--enable-prefix-caching / --no-enable-prefix-caching	Sempre.
Sem CUDA graphs	--enforce-eager	Sempre.
Precisão	--dtype	Só em modelo sem quantização.
GPUs (paralelismo de tensor)	--tensor-parallel-size	2 ou mais GPUs NVIDIA; o número precisa dividir as cabeças de atenção.
Paralelismo de especialistas	--enable-expert-parallel	Modelo MoE com paralelismo de tensor maior que 1.
Decodificação especulativa	--speculative-config	ngram sempre; mtp quando o config.json indica camadas MTP. O launcher monta o JSON.

Strata e PolyStrata

Contexto, quantização e GPU vêm da configuração strata-*.json e aparecem só para leitura. Os campos editáveis (**GPU(s)** --gpu, **Fila máxima** --max-queue, **Descarregar após ociosidade** --idle-unload e **VRAM livre mínima** --min-free-vram-mib) só ficam ativos se o serve/server.py instalado aceitar a opção; o launcher confere no próprio arquivo, porque esses projetos mudam rápido.

Sugestão, valores automáticos e autotuning

- **Aplicar sugestão inicial** preenche os campos pela RAM, CPU, GPU/VRAM e pelos dados do modelo. No GGUF, liga draft-mtp quando o modelo tem camadas MTP. No vLLM, calcula contexto, uso de memória e paralelismo de tensor pelo config.json e pela VRAM, e pergunta se você quer rodar o autotuning. Não existe no Strata.
- **Calcular valores automáticos** (só GGUF) roda o llama-fit-params, que vem no pacote oficial, e mostra nos campos em Auto o que o -fit escolheria: camadas na GPU, contexto, proporção por GPU e quantas camadas de especialistas ficariam na CPU. Roda sozinho ao abrir o perfil de um GGUF e leva alguns segundos.
- **Otimizar automaticamente** procura os valores mais rápidos dentro do **Tempo** escolhido (5, 10, 20 ou 30 minutos) e da **Prioridade**: **Geração** (padrão; tokens/s de geração), **Prompt longo** (tokens/s de prompt) ou **Equilíbrio**. **Cancelar** para o teste e aplica o melhor até ali. O resultado vai para os campos, mas o perfil só é salvo quando você clicar em **Salvar**. **Medições do autotuning** mostra cada tentativa, inclusive as que falharam e o motivo.
- **llama.cpp:** primeiro o llama-fit-params define o teto de memória. Depois o llama-bench testa, um de cada vez, especialistas na CPU (MoE), Flash Attention, cache K/V (f16, q8_0, q4_0), microbatch, threads (quando a CPU trabalha) e divisão entre GPUs. Se sobrar tempo, um llama-server curto mede o que o llama-bench não mede: decodificação especulativa (com a taxa de aceitação do rascunho), threads de prompt e cache de especialistas na GPU. O que não cabe na memória ou não é suportado (cache V quantizado sem Flash Attention, por exemplo) sai da lista. Um valor só troca o atual com ganho de pelo menos 2%.
- **vLLM:** cada tentativa sobe o servidor de novo (no Windows, um contêiner) e recarrega o modelo, então leva vários minutos por tentativa: prefira 20 ou 30 minutos. Testa uso de memória da GPU, tipo do cache KV, tokens por lote e decodificação especulativa.
- **Strata/PolyStrata:** não há autotuning.

Depois que o servidor sobe, a linha **Em uso**, abaixo do endpoint, mostra os valores reais: contexto, camadas na GPU e sessões no llama.cpp (lidos do /props e do registro); tamanho do cache KV e concorrência máxima no vLLM.

Perfis salvos antes da versão 0.9.0

Perfis antigos continuam funcionando com os mesmos valores. Na primeira abertura da 0.9.0 eles ganham version: 2; -ngl auto e -fa auto passam a aparecer como Auto, que tem o mesmo efeito. Os campos novos começam em Auto.

Análise pelo modelo

Na tela **Modelos**, **Copiar prompt** copia um pedido de análise do hardware e dos ajustes; **Analisar com modelo selecionado** envia esse pedido pela conversa integrada. A resposta é só uma recomendação e não altera nada sozinha.

11. Acesso pela rede e compartilhamento

Em **Acesso e sessões**, o **Endereço de acesso** lista:

- **Somente este computador (127.0.0.1):** sem chave; só programas desta máquina acessam.
- **Rede interna:** endereços privados (10.x, 172.16-31.x, 192.168.x). Exige a chave da API.
- **Tailscale:** endereços 100.64.0.0/10 da sua tailnet. Exige a chave e é a opção preferida, porque o tráfego é cifrado.

Na rede, use **Mostrar chave**, **Copiar chave** ou **Nova chave** (pare o servidor antes de gerar outra). Com o servidor ativo em LAN ou Tailscale, o painel **Conectar aplicativo** oferece:

- **Copiar link com acesso:** abre a interface no navegador de outro computador sem digitar a chave.
- **Copiar Bash/PowerShell com acesso** e **Copiar configuração com acesso:** levam uma credencial temporária.
- **Revogar acesso compartilhado:** invalida tudo o que foi compartilhado e derruba as conexões abertas. Parar o servidor ou fechar o launcher também encerra o compartilhamento.

Quem recebe um link ou comando com acesso pode usar o modelo, mas não acessa os arquivos do seu computador. No Windows, a seção **Firewall do Windows** cria a regra de entrada para a porta ativa. A regra continua depois que o servidor para; remova-a em **Configurações > 07 · Firewall do Windows**. Detalhes de portas, chaves, clientes e firewall estão no **Guia de integração**.

12. Dados, atualização e desinstalação

As preferências e os perfis ficam em settings.json, na pasta de dados do aplicativo. Nessa mesma pasta ficam a chave da API (server-api-key.txt), os motores baixados (engines/) e os pacotes de atualização (updates/). O Guia de diagnóstico mostra o caminho em cada sistema. Os modelos ficam nas pastas que você escolheu.

Atualizar o aplicativo

Ao abrir e uma vez por dia, o aplicativo consulta a release mais recente no GitHub. Quando há versão nova, o cabeçalho mostra **vX.Y.Z disponível**. Abra **Configurações > 05 · Atualizações** para ver as notas. Desmarque **Verificar atualizações ao iniciar** para desligar a consulta e use **Verificar agora** quando quiser.

Baixar e instalar (ou **Baixar atualização**) baixa o pacote do seu sistema e o confere com o SHA256SUMS.txt da release antes de usar.

- **Windows:** o aplicativo pede para encerrar o servidor e a conversa, abre o instalador e fecha.
- **macOS:** a imagem de disco é aberta; arraste o app para **Aplicativos** e substitua a versão atual.
- **Linux AppImage:** a nova AppImage é salva ao lado da atual, já executável.
- **Linux DEB/RPM:** o pacote verificado é mostrado na pasta, com o comando sudo apt install ou sudo dnf install. Feche o app antes de instalar.

Motores, modelos, perfis e a chave da API são mantidos. Os instaladores ainda não têm assinatura digital: a verificação protege contra download corrompido, não contra uma release adulterada.

Desinstalar

Pare o servidor e feche o aplicativo antes. No Windows, use **Configurações > Aplicativos instalados**; o desinstalador pede para fechar o app se ele ainda estiver aberto e **preserva** modelos, perfis e motores. No Linux, remova o DEB/RPM pelo gerenciador ou apague a AppImage. No macOS, mova o app para o Lixo. Em todos os sistemas, os dados do usuário e os modelos ficam para você revisar e apagar manualmente, se quiser. O vLLM no Windows deixa a imagem vllm/vllm-openai no Docker Desktop (remova por lá), e a instalação do Strata/PolyStrata continua na pasta dela.

13. Créditos

O llama.cpp foi iniciado por Georgi Gerganov e é mantido pelos colaboradores da ggml-org, sob licença MIT. O vLLM é do projeto vllm-project, sob licença Apache 2.0. O Strata foi criado por Niko1221 e colaboradores, sob licença MIT. O PolyStrata foi criado por VecSzn e colaboradores, baseado no Strata, sob licença MIT.

Este launcher independente foi desenvolvido para Célio com Codex, com base no launcher WinForms anterior, e está sob **licença MIT**, com copyright de Célio. Ele não redistribui esses motores: baixa o pacote oficial do llama.cpp, instala o pacote ou a imagem oficial do vLLM e só inicia instalações do Strata e do PolyStrata feitas pelos instaladores oficiais. Os autores desses projetos não participam nem endossam este aplicativo. Cada modelo tem licença própria.